

Математические основы алгоритмов, осень 2020 г.
 Лекция 7: Строковые алгоритмы. Полиномиальное
 хэширование: алгоритм Рабина–Карпа, наибольшая общая
 подстрока, самый длинный палиндром. Префиксные
 деревья. Конечные автоматы. Алгоритм
 Кнута–Морриса–Пратта, алгоритм Ахо–Корасик, их
 реализация на конечном автомате. Упрощённое суффиксное
 дерево и его построение*

Александр Охотин

8 июня 2021 г.

Содержание

1	Полиномиальное хэширование	2
1.1	Хэш-функции для строк	2
1.2	Алгоритм Рабина–Карпа	2
1.3	Наибольшая общая подстрока	3
1.4	Самый длинный палиндром	4
2	Алгоритм Кнута–Морриса–Пратта	4
3	Поиск с помощью конечных автоматов	6
3.1	Конечные автоматы	7
3.2	Конечный автомат, исполняющий алгоритм Кнута–Морриса–Пратта	8
4	Представление множеств строк	9
4.1	Лексикографический порядок	9
4.2	Структуры данных для хранения множества строк	9
5	Поиск нескольких строк одновременно: алгоритм Ахо–Корасик	11
5.1	Обобщение префиксной функции	11
5.2	Построение функции π	11
5.3	Конечный автомат, исполняющий алгоритм Ахо–Корасик	13

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.

6 Суффиксные деревья	14
6.1 Упрощённое суффиксное дерево	15
6.2 Построение упрощённого суффиксного дерева	16

1 Полиномиальное хэширование

Строки над алфавитом Σ , подстроки, префиксы, суффиксы.

Задача поиска в строке. Дана длинная строка («текст») $w = a_1 \dots a_n$, и короткая искомая строка («шаблон») $x = b_1 \dots b_m$. Требуется найти все вхождения x в w в качестве подстроки, то есть, все смещения s , для которых подстрока $w_s = a_{s+1} \dots a_{s+m}$ совпадает с $b_1 \dots b_m$.

«Наивный» алгоритм: сравнивать все m символов для каждого s , время $O(mn)$. В худшем случае достигается, пример: $x = 0^{m-1}1$ и $w = 0^{2m-1}1$, всего m^2 сравнений.

Но можно искать быстрее.

1.1 Хэш-функции для строк

Пусть нужно быстро сравнивать строки на равенство. Каждой строке ставится в соответствие число: $w \rightsquigarrow h(w)$ — например, 32- или 64-битное. Это число будет храниться вместе со строкой. Тогда, если нужно сравнить две строки, сперва сравниваются соответствующие им числа. Если они отображаются в разные числа, то они точно разные. Если же они отображаются в одно и то же число, то они могут быть одинаковыми, а могут быть и разными — и тогда нужно сравнивать их посимвольно.

Но если хэш-функция подобрана удачно, то случайные совпадения значений хэш-функций разных строк будут встречаться крайне редко, и потому на посимвольное сравнение строк будет в среднем затрачиваться пренебрежимо мало времени.

Как лучше определить хэш-функцию?

Самое простое, что можно придумать: сложить коды всех символов, взять сумму по модулю 2^{32} . Недостаток: распределение кодов неравномерно! Поэтому и сумма будет распределена неравномерно, и строки, совпадающие с точностью до порядка символов, получат одинаковые значения. Это совсем плохо.

Полиномиальное хэширование: берётся некоторое основание степени p . Пусть $w = a_1 \dots a_\ell$ — строка. Тогда используется сумма $\sum_{i=1}^{\ell} a_i \cdot p^{\ell-i}$, взятая по некоторому модулю M .

1.2 Алгоритм Рабина–Карпа

Алгоритм поиска подстроки $x = b_1 \dots b_m$ в строке $w = a_1 \dots a_n$, предложенный Карпом и Рабином [1987], основан на полиномиальном хэшировании степени $m - 1$: сперва вычисляется значение хэш-функции для искомой строки, а затем для всех m -символьных подстрок данного текста последовательно вычисляется значение их хэш-функции. Когда значение для подстроки совпадает со значением для искомой строки, алгоритм проводит прямое сравнение символов, как в «наивном» алгоритме.

Значение хэш-функции для искомой строки: $X = \sum_{i=1}^m b_i \cdot p^{m-i}$ по модулю q .

Значение хэш-функции для подстроки со смещением s : $W_s = \sum_{i=1}^m a_{s+i} \cdot p^{m-i}$ по модулю q .

Алгоритм сперва вычисляет X , а затем последовательно вычисляет W_s для s от 0 до $n - m$. Удобство этой хэш-функции в том, что каждое следующее значение W_{s+1} можно вычислить на основе значения W_s за $O(1)$ шагов. Для этого достаточно заметить, что числа



Рис. 1: Майкл Рабин (род. 1931) и Ричард Карп (род. 1935).

pW_s и W_{s+1} отличаются всего на два слагаемых.

$$\begin{aligned} pW_s &= a_{s+1}p^m + a_{s+2}p^{m-1} + \dots + a_{s+m}p \\ W_{s+1} &= a_{s+2}p^{m-1} + \dots + a_{s+m}p + a_{s+m+1} \end{aligned}$$

Отсюда W_{s+1} получается из W_s по следующей формуле (вся арифметика — по модулю q).

$$W_{s+1} = p \cdot W_s - a_{s+1} \cdot p^m + a_{s+m+1}$$

Сложность: $\Theta(m)$ на подготовку, и затем в худшем случае $O(mn)$, если хэш-функция выдаст одинаковые значения для всех подстрок. Это неизбежно, если, например, все подстроки одинаковы, то есть, $w = a^n$ и $x = a^m$.

Но в среднем случае получается время работы $\Theta(n)$.

1.3 Наибольшая общая подстрока

Полиномиальное хэширование позволяет легко построить относительно неплохие, пусть и не самые оптимальные алгоритмы для решения ряда задач.

Задача 1 (Наибольшая общая подстрока). Дано: $u = a_1 \dots a_m$ и $v = b_1 \dots b_n$. Найти самую длинную строку x , для которой $u = u_1xu_2$ и $v = v_1xv_2$.

Наивное решение: для каждой пары позиций найти самую длинную подстроку, время $O((m+n)^3)$.

Как можно сделать это лучше? Сперва с помощью полиномиального хэширования надо научиться отвечать на вопрос «Есть ли общая подстрока длины ℓ ?» за время $O((m+n) \log(m+n))$. Для этого находятся значения хэш-функции для всех подстрок u длины ℓ , размещаются в двоичном дереве поиска — время $m \log m$. То же самое — для v , время $n \log n$. Далее исследуются совпадающие значения.

Наконец, используется двоичный поиск по ℓ . Его можно записать в виде рекурсивной процедуры, отвечающей на вопрос «Найти длину наибольшей общей подстроки, если известно, что её длина не меньше, чем ℓ_1 , и строго меньше, чем ℓ_2 ?» (алгоритм 1). А можно то же самое сделать и без рекурсии.

Алгоритм 1 Нахождение длины наибольшей общей подстроки двоичным поиском

процедура $f(\ell_1, \ell_2)$

```
1: if  $\ell_2 - \ell_1 = 1$  then
2:   return  $\ell_1$ 
3:  $k = \lfloor \frac{\ell_1 + \ell_2}{2} \rfloor$ 
4: if есть общая подстрока длины  $k$  then
5:   return  $f(k, \ell_2)$ 
6: else
7:   return  $f(\ell_1, k)$ 
```

1.4 Самый длинный палиндром

Обращение строки — запись всех символов в обратном порядке: $(a_1 a_2 \dots a_{n-1} a_n)^R = a_n a_{n-1} \dots a_2 a_1$. Если $u = u^R$, такая строка называется *палиндромом*.

Задача 2 (Самый длинный палиндром). По данной строке $w = a_1 \dots a_n$ найти самую длинную её подстроку-палиндром.

Наивный алгоритм: искать вокруг каждого символа на длину вплоть до $\frac{n}{2}$, время $O(n^2)$.

Улучшенный алгоритм основан на решении подзадачи: «Есть ли подстрока-палиндром данной длины ℓ ?» После того, как будет построен алгоритм для решения подзадачи, останется использовать двоичный поиск по ℓ .

Задачу поиска палиндромов заранее заданной длины ℓ можно переформулировать так. Подстрока $a_{i+1} \dots a_{i+\ell}$ строки w — палиндром, если она совпадает со строкой $a_{i+\ell} \dots a_{i+1}$, которая в свою очередь встречается в строке w^R , начиная с позиции $n - i - \ell + 1$. Стало быть, нужно проверять на равенство подстроки длины ℓ , начинающиеся с позиции $i + 1$ в w и с позиции $n - i - \ell + 1$ в w^R . Это удобно сделать, вычислив значения полиномиального хэша для всех подстрок w и w^R длины ℓ , а затем для каждого i сравнить хэш w для позиции $i + 1$ и хэш w^R для позиции $n - i - \ell + 1$. Для каждого совпадения придётся проверять подстроки на равенство посимвольно, но если хэш-функция не подведёт, это будет требоваться очень редко, так что среднее время будет линейным.

Двоичный поиск по ℓ даст время $O(n \log n)$.

2 Алгоритм Кнута–Морриса–Пратта

Улучшение «наивного» поиска, не использующее хэширование и работающее быстро на любых входных данных. Пусть в строке $w = ababaab$ ищется подстрока $x = abaa$. Сперва $w_1 = abab$ сравнивается с $abaa$ и выясняется, что они различны в третьем символе. Имеет ли после этого смысл сравнивать $w_2 = baba$ с $abaa$? Никакого, потому что раз предыдущая подстрока $w_1 = abab$ отличается от x лишь в четвёртом символе, а первый и второй символы x различны, это значит, что первый символ w_2 не совпадает с первым символом x . Эти сведения можно получить исключительно из самой строки x и количества совпадающих первых символов у w_1 и x (их 3). Поэтому алгоритм может сразу переходить к сравнению w_3 с x . Более того, поскольку первый и третий символ x совпадают, из результата первого сравнения уже известно, что первый символ w_3 совпадает с первым символом x , и потому их можно сравнивать, начиная со вторых символов.

Подобные рассуждения можно автоматизировать. Алгоритм Кнута–Морриса–Пратта [1977] (Knuth–Morris–Pratt, KMP), читает строку w слева направо, и после чтения i символов w



Рис. 2: Дональд Кнут (род. 1938), Джеймс Моррис (род. 1941), Вон Пратт (род. 1944).

он помнит длину j самого длинного префикса строки x , на который заканчивается $a_1 \dots a_i$ — то есть, наибольшее число j , для которого верно $a_{i-j+1} \dots a_i = b_1 \dots b_j$.

При чтении очередного символа a_{i+1} алгоритм сравнивает его с b_{j+1} , и если эти символы равны, это значит, что префикс x длины j продлевается на один символ. Если же $a_{i+1} \neq b_{j+1}$, то самый длинный префикс x , на который заканчивается строка $a_1 \dots a_{i+1}$, будет содержать строго меньше, чем $j + 1$ символов. Чтобы найти длину этого префикса, нужно рассмотреть *следующий по длине префикс x на который заканчивается $a_1 \dots a_i$* , и попытаться продлевать уже его.

Чтобы иметь возможность находить длину такого префикса, алгоритм заранее строит по данной искомой строке x определённую структуру данных — *префиксную функцию*.

Определение 1. Префиксная функция для строки $x = b_1 \dots b_m$ — это функция $\pi: \{1, \dots, m\} \rightarrow \{0, \dots, m-1\}$, которая, для всякого префикса $b_1 \dots b_i$ строки x выдаёт длину наибольшего суффикса подстроки $b_1 \dots b_i$, который также является префиксом x .

$$\pi(i) = \max\{j \mid j < i, b_1 \dots b_j = b_{i-j+1} \dots b_i\}$$

В вышеприведённом примере для строки $x = abab$, значение префиксной функции для $i = 3$ равно $\pi(3) = 1$, и руководствуясь этим, алгоритм может перескочить к сравнению вторых символов w_3 и x .

Повторно применяя функцию π , можно получить все префиксы x , являющиеся суффиксами $b_1 \dots b_i$, и алгоритм будет их перебирать, пока не найдёт такой, который можно продолжить следующим символом текста.

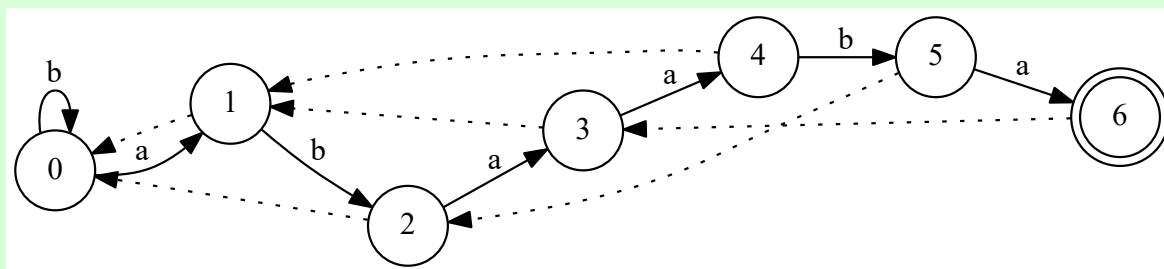
Алгоритм поддерживает следующий инвариант: в каждой i -й итерации внешнего цикла, перед выполнением строчки 7, самый длинный префикс x , являющийся суффиксом $a_1 \dots a_i$ — это строка $b_1 \dots b_j$.

Пример 1. Функция π для строки $x = abaaba$ принимает следующие значения: $\pi(1) = 0$, $\pi(2) = 0$, $\pi(3) = 1$, $\pi(4) = 1$, $\pi(5) = 2$, $\pi(6) = 3$.

Алгоритм 2 Алгоритм Кнута–Морриса–Пратта

На входе: строка $w = a_1 \dots a_n$.

```
1:  $j = 0$ 
2: for  $i = 1$  to  $n$  do
3:   while  $b_{j+1} \neq a_i$  и  $j > 0$  do
4:      $j = \pi(j)$ 
5:   if  $b_{j+1} = a_i$  then
6:      $j = j + 1$ 
7:   if  $j = m$  then
8:     подстрока найдена по смещению  $i - m$ 
9:      $j = \pi(j)$ 
```



Лемма 1. *Имея готовую таблицу значений функции π , алгоритм КМР работает за время $\Theta(n)$.*

Доказательство. На первый взгляд, строчка 4 при неудачном стечении обстоятельств может выполняться mn раз. Но это не так. Всякий раз, когда выполняется строчка 4, значение j уменьшается, но ниже нуля оно никогда не становится. Поскольку за всё время работы алгоритма значение j увеличивается не более чем на n (в строке 6), уменьшиться больше чем n раз оно не сможет. $\square$

Осталось научиться быстро строить значения префиксной функции. Это делает алгоритм 3, который, действуя в точности так же, как алгоритм 2, ищет вхождения строки x в саму себя. На самом деле он находит вхождения *префиксов* строки x в неё саму — и это и есть искомые префиксы-суффиксы.

3 Поиск с помощью конечных автоматов

Сколько памяти использует алгоритм Кнута–Морриса–Пратта? Если пренебречь зависимостью от длины искомой строки, то алгоритм использует $\log n$ битов, необходимых для хранения переменной i . Однако всё, что делается с этой переменной — это чтение символа a_i и увеличение значения i на единицу. За счёт этого входные символы последовательно

Алгоритм 3 Построение значений префиксной функции

```
1:  $\pi(1) = 0$ 
2:  $j = 0$ 
3: for  $i = 2$  to  $m$  do
4:   while  $b_{j+1} \neq b_i$  и  $j > 0$  do
5:      $j = \pi(j)$ 
6:   if  $b_{j+1} = b_i$  then
7:      $j = j + 1$ 
8:    $\pi(i) = j$ 
```

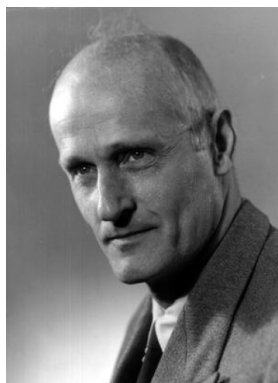


Рис. 3: Стивен Клини (1909–1994).

читаются один за другим — и только. А в остальном, алгоритм использует $O(1)$ памяти — то есть, *конечный её объём*, не зависящий от длины просматриваемой строки.

Работу алгоритма Кнута–Морриса–Пратта можно формализовать в терминах важной теоретической модели — *конечного автомата*.

3.1 Конечные автоматы

Определение 2 (Клини). *Детерминированный конечный автомат* (*deterministic finite automaton, DFA*) — пятёрка $\mathcal{A} = (\Sigma, Q, q_0, \delta, F)$, со следующим значением компонентов.

- Σ — алфавит (конечное множество).
- Q — конечное множество состояний.
- $q_0 \in Q$ — начальное состояние.
- Функция переходов $\delta: Q \times \Sigma \rightarrow Q$. Если автомат находится в состоянии $q \in Q$ и читает символ $a \in \Sigma$, то его следующее состояние — $\delta(q, a)$.
- Множество **принимаящих состояний** $F \subseteq Q$.

Для всякой входной строки $w = a_1 \dots a_\ell$, где $\ell \geq 0$ и $a_1, \dots, a_\ell \in \Sigma$, вычисление — последовательность состояний $p_0, p_1, \dots, p_{\ell-1}, p_\ell$, где $p_0 = q_0$, и всякое следующее состояние p_i , где $i \in \{1, \dots, \ell\}$, однозначно определено как $p_i = \delta(p_{i-1}, a_i)$.

$$p_0 \xrightarrow{a_1} p_1 \xrightarrow{a_2} \dots \xrightarrow{a_{\ell-1}} p_{\ell-1} \xrightarrow{a_\ell} p_\ell$$

Строка **принимается**, если последнее состояние p_ℓ принадлежит множеству F — иначе **отвергается**.

Множество строк, **распознаваемое** автоматом, обозначаемое через $L(\mathcal{A})$ — состоит из всех строк, которые он принимает.

Множество строк в теоретической информатике называют *формальным языком* (formal language) или просто *языком* (language).

Дополнительное обозначение: если DFA начинает вычисление в состоянии $q \in Q$ и читает строку $w = a_1 \dots a_\ell$, то его состояние после её прочтения обозначается через $\delta(q, w)$. Формально, определение функции переходов расширяется до $\delta: Q \times \Sigma^* \rightarrow Q$, и определяется как $\delta(q, \varepsilon) = q$ и $\delta(q, aw) = \delta(\delta(q, a), w)$. В этих обозначениях, язык, распознаваемый DFA, кратко определяется так.

$$L(\mathcal{A}) = \{ w \in \Sigma^* \mid \delta(q_0, w) \in F \}$$

Реализация на компьютере: таблица значений функции δ хранится в массиве, при чтении каждого символа необходимо один раз обратиться к таблице и изменить одну переменную — состояние DFA.

3.2 Конечный автомат, исполняющий алгоритм Кнута–Морриса–Пратта

Теорема 1. Для всякой строки $x = b_1 \dots b_m$ над алфавитом Σ существует DFA с $m + 1$ состояниями, распознающий множество Σ^*x всех строк, заканчивающихся на x , и этот DFA можно построить за время $\Theta(|\Sigma| \cdot m)$.

Доказательство. Сперва строится префиксная функция π для искомой строки x .

Автомат будет читать строку w посимвольно, слева направо — в точности как это делает алгоритм КМР. За один переход автомат должен проделать целую i -ю итерацию алгоритма. Состояния автомата соответствуют значениям переменной j в алгоритме КМР: после чтения строки w автомат должен перейти в такое состояние j , что $a_1 \dots a_j$ — это самый длинный суффикс строки w .

$$Q = \{0, 1, \dots, m\}$$

В начальном состоянии автомат не успел ещё прочитать ни одного символа входной строки.

$$q_0 = 0$$

Переход в каждом состоянии описывает поведение алгоритма КМР. Определение даётся индуктивно по j , сперва для случая $j = 0$.

$$\delta(0, a) = \begin{cases} 0, & b_1 \neq a \\ 1, & b_1 = a \end{cases}$$

Далее, функция последовательно определяется для всех i от 1 до $m - 1$.

$$\delta(j, a) = \begin{cases} \delta(\pi(j), a), & \text{если } b_{j+1} \neq a \\ j + 1, & \text{если } b_{j+1} = a \end{cases}$$

В случае $j = t$ автомат повторяет переходы из состояния $\pi(t)$.

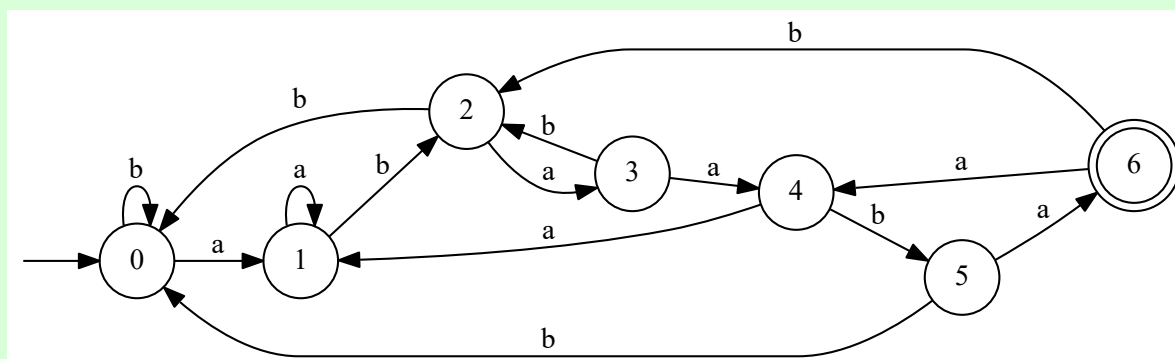
$$\delta(t, a) = \delta(\pi(t), a)$$

Наконец, принимающие состояния — это те, в которых только что было закончено чтение подстроки x , и алгоритм КМР готов об этом сообщить.

$$F = \{t\}$$

□

Пример 2 (продолжение примера 1). Функция π для строки $w = abaaba$ принимает значения $\pi(1) = 0$, $\pi(2) = 0$, $\pi(3) = 1$, $\pi(4) = 1$, $\pi(5) = 2$, $\pi(6) = 3$. По ней строится следующий конечный автомат.



4 Представление множеств строк

4.1 Лексикографический порядок

Отношение порядка на множестве строк: *лексикографический порядок*. Сперва алфавит $\Sigma = \{a_1, \dots, a_k\}$ упорядочивается как $a_1 < \dots < a_k$. Затем всякие две строки сравниваются по первому различному символу. А именно, если $u = xay$, $v = xbz$ и $a < b$, то $u < v$; если же $u = x$ и $v = xbz$, то $u < v$.

$$\begin{aligned} a &< \text{абажур} < \text{абак} < \text{аббат} < \dots < \text{ящур} \\ a &< \text{aback} < \text{abaft} < \text{abandon} < \dots < \text{zythum} \end{aligned}$$

4.2 Структуры данных для хранения множества строк

Как вообще можно хранить множество строк? Во-первых, **в виде списка**. Если список содержит n строк длины ℓ каждая, то безуспешный поиск может потребовать около $n\ell$ действий!

С помощью лексикографического порядка, или любого другого порядка на множестве строк, можно задать **двоичные деревья поиска**. Если сбалансированное двоичное дерево поиска содержит n строк длины ℓ каждая, то каждая операция может потребовать времени $\ell \log n$.

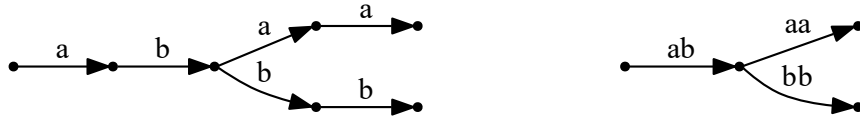


Рис. 4: (слева) Префиксное дерево и (справа) компактное префиксное дерево для множества $\{abaa, abbb\}$.

Однако в большинстве случаев удобнее использовать особую структуру данных для хранения множеств строк — **префиксное дерево**, впервые предложенное де ла Бриан-де [1959]. Префиксное дерево (prefix tree; более распространенное английское название — trie, от reTRIEval, но произносится, как try¹) — это корневое дерево с дугами, помеченными символами алфавита. Дуги, исходящие из всякой вершины, должны быть помечены различными символами. Каждая вершина соответствует некоторой строке, которая хранится неявно, в виде последовательности дуг на пути к ней. Также всякая вершина хранит один бит информации, определяющий, принадлежит ли эта строка хранимому множеству; вместе с битом можно хранить любые дополнительные значения, сопоставленные этой строке. Корень соответствует пустой строке. Если вершина соответствует строке w , и исходящая из неё дуга помечена символом $a \in \Sigma$, то эта дуга идёт в вершину, соответствующую строке wa .

Все операции с любой данной строкой длины ℓ (поиск, вставка, удаление) выполняются за время $O(\ell)$, не зависящее от числа элементов в хранимом множестве.

При реализации этой структуры данных надо иметь в виду следующее. Во-первых, если $|\Sigma| = 256$, то придётся хранить 256 указателей в каждой вершине, это много. А если алфавит — Unicode? Как только размер алфавита начинает мешать, можно разбить каждый символ на несколько подсимволов: например, если $|\Sigma| = 256$, то можно кодировать каждый символ двумя символами из 16-символьного алфавита. Другая возможность — сделать в каждой вершине префиксного дерева небольшое двоичное дерево поиска для символов.

Во-вторых, в тех случаях, когда хранимые в префиксном дереве строки часто будут содержать длинные пути без ветвления, используется *компактное префиксное дерево* (compact prefix tree, radix tree) — то же самое, но дуги помечены не отдельными символами, а непустыми строками. Эти непустые строки задаются парами указателей на последовательности символов, размещённые в каком-то массиве. Компактное префиксное дерево для множества $\{abaa, abbb\}$ будет состоять из 4 вершин: корня, из которого исходит дуга ab во внутреннюю вершину, из которой в свою очередь исходят дуги aa и bb в два листа.

Лемма 2. В компактном префиксном дереве для множества из n элементов не более чем n внутренних вершин.



Рис. 5: Альфред Ахо (род. 1941), Маргарет Корасик (род. 1942).

5 Поиск нескольких строк одновременно: алгоритм Ахо–Корасик

Пусть для данного текста $w = a_1 \dots a_n$ требуется найти все вхождения не одной строки, а всех строк из конечного множества $K = \{x_1, \dots, x_k\}$.

Алгоритм Кнута–Морриса–Пратта решал частный случай этой задачи для $k = 1$. При этом на каждом шаге алгоритм помнил, сколько первых символов строки x_1 находятся в конце прочитанной части входной строки. Иными словами, алгоритм помнил самый длинный только что прочитанный префикс строки x_1 .

Алгоритм Ахо–Корасик [1975] будет помнить самый длинный только что прочитанный префикс одной из строк $x_1, \dots, x_k$ — точнее сказать, самую длинную такую строку u , что u — префикс какой-то строки из K , и алгоритм только что прочитал u .

5.1 Обобщение префиксной функции

Когда алгоритм вслед за префиксом u читает следующий символ a , он пытается прочитать этот символ в префиксном дереве. Если префикс ua существует, то алгоритм его запоминает и продолжает работу. Если же префикса ua нет, алгоритм должен откатиться на следующий по длине префикс *одной из строк* $x_1, \dots, x_k$ и попробовать прочитать u из него.

Такой откат реализуется обобщением префиксной функции π из алгоритма КМР. Обобщённую функцию можно обозначить через $\pi: \text{prefixes}(K) \rightarrow \text{prefixes}(K)$. В дереве её можно обозначить пунктирной линией, ведущей из одной вершины в другую, расположенную ближе к корню. Прежде чем формулировать сам алгоритм Ахо–Корасик, стоит рассмотреть построение этой функции.

Алгоритм 4 использует функцию π , чтобы найти все вхождения всех строк $x_1, \dots, x_k$ в поданную на вход строку w .

5.2 Построение функции π

Алгоритм последовательно вычисляет значение π , продвигаясь от коротких суффиксов к длинным. Поддерживается очередь вершин для обработки, содержащая вершины, для которых функция π уже определена, а для их потомков ещё не определена.

Сперва алгоритм определяет функцию π для всех односимвольных строк $a \in \Sigma$, находящихся в дереве. Для каждой из них самый длинный суффикс — это пустая строка: $\pi(a) = \varepsilon$. Все эти вершины заносятся в очередь.

¹Русские олимпиадные программисты называют эту структуру данных своим особым жаргонным словечком; его, как и всякий жаргон небольшого сообщества, пожалуй, всё же не стоит употреблять на людях, чтобы не осрамиться.

Алгоритм 4 Алгоритм Ахо–Корасик

На входе: строка $w = a_1 \dots a_n$.

```
1:  $u = \varepsilon$  /* корень префиксного дерева */
2: for  $i = 1$  to  $n$  do
3:   while в дереве нет  $ua_i$  и  $u \neq \varepsilon$  do
4:      $u = \pi(u)$ 
5:   if в дереве есть  $ua_i$  then
6:      $u = ua_i$ 
7:   if вершина  $u$  в дереве помечена then
8:     подстрока найдена по смещению  $i - |u|$ 
```

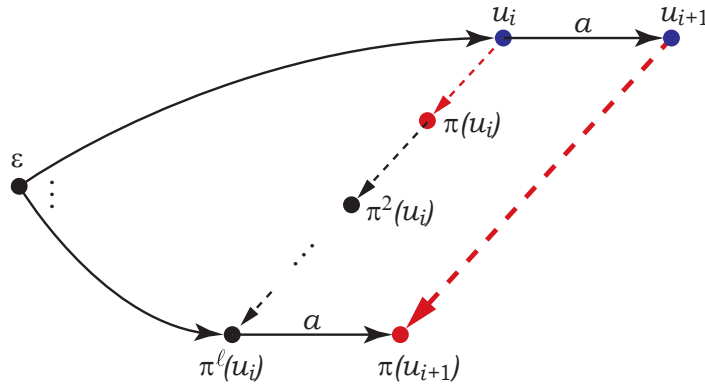


Рис. 6: Вычисление значения $\pi(u_{i+1})$, где $u_{i+1} = u_i a$, в алгоритме Ахо–Корасик.

Затем на каждом шаге алгоритм извлекает из очереди вершину, соответствующую некоторой строке u , для которой самый длинный суффикс $\pi(u)$ уже определён. Рассматривается всякое её продолжение по символу $a \in \Sigma$, находящееся в дереве. Делается попытка продлить самый длинный суффикс $v = \pi(u)$ символом a : если в префиксном дереве есть такое продолжение va , то значение $\pi(ua)$ и будет на него указывать. Кроме того, если в va распознаётся одна из строк $x_i \in K$, то она же должна распознаваться и в ua .² Если же такого продолжения нет, то будут последовательно перебираться вершины $\pi(u)$, $\pi^2(u) = \pi(\pi(u))$ и т.д., пока среди них не найдётся такая вершина $\pi^\ell(u)$, для которой в префиксном дереве есть переход по a — и значение $\pi(ua)$ будет указывать на строку $\pi^\ell(u)a$. Если же алгоритм придёт в корень и перехода по a нет и оттуда, то ссылка $\pi(ua)$ направится в корень. Наконец, вершина ua помещается в конец очереди.

Линейность времени работы полученного алгоритма 5 отнюдь не очевидна. Для каждой очередной вершины ua последовательность префиксных ссылок может иметь линейную длину, и всего вершин линейное число. Тем не менее, доказывается, что суммарное количество переходов по префиксным ссылкам ограничено линейной функцией.

Лемма 3. Алгоритм 5 работает за линейное время от суммарной длины искомых строк.

Доказательство. Для каждой из искомых строк $x = a_1 \dots a_n \in K$, рассматривается вычисление значения π для всех префиксов этой строки, $u_i = a_1 \dots a_i$. Утверждается, что сум-

²На рис. 7(нижнем) это происходит при добавлении ссылки $\pi(abb) = bb$: поскольку строка bb распознаётся в вершине bb , она должна распознаваться и в вершине abb .

Алгоритм 5 Алгоритм Ахо–Корасик: построение функции π

Дано: множество искоемых строк $K = \{x_1, \dots, x_k\}$.

```
1: построить префиксное дерево для  $K$ 
2: for all  $a \in \Sigma$  do
3:     if в дереве есть вершина  $a$  then
4:         добавить её в очередь
5:         определить  $\pi(a) = \varepsilon$ 
6:     else
7:         добавить петлю по  $a$  в корне
8: while в очереди есть вершины do
9:     извлечь вершину  $u$  из очереди
10:    for all  $a \in \Sigma$  do
11:        if в дереве есть вершина  $ua$  then
12:            пусть  $v = \pi(u)$ 
13:            while в дереве нет вершины  $va$  do
14:                 $v = \pi(v)$ 
15:            определить  $\pi(ua) = va$ 
16:            пометить, что все строки  $x \in K$ , найденные в  $va$ , найдены и в  $ua$ 
17:            добавить вершину  $ua$  в конец очереди
```

марное число выполнений строчки 14, для всех i , ограничено сверху числом n . Доказательство по сути повторяет доказательство аналогичного свойства алгоритма Кнута–Морриса–Пратта: рассматривается последовательность $|\pi(u_1)|, |\pi(u_2)|, \dots, |\pi(u_n)|$, и утверждается, что значение $|\pi(u_i)|$ увеличивается не более n раз. Общее же число шагов вычисления ограничено *количеством уменьшений* этого значения.

Пусть $v_i = \pi(u_i)$ — вычисленное значение для каждого префикса. Вычисляя очередной префикс v_{i+1} , алгоритм проходит последовательность строк $\pi(u_i), \pi^2(u_i), \dots, \pi^{\ell_i}(u_i)$; каждое применение функции π уменьшает длину строки не менее чем на единицу, а продление $\pi^{\ell_i}(u_i)$ на один символ может увеличить длину на единицу. Поэтому число выполнений строчки 14 на этом шаге не превосходит $|v_{i+1}| - |v_i| + 1$. Сумма по всем i даёт верхнюю оценку n . $\square$

5.3 Конечный автомат, исполняющий алгоритм Ахо–Корасик

Подобно алгоритму Кнута–Морриса–Пратта, алгоритм Ахо–Корасик естественным образом реализуется на конечном автомате, читающем строку w слева направо и определяющем все вхождения строк из конечного множества K в w .

Надо сказать, что префиксное дерево само по себе — это по сути конечный автомат, распознающий хранимое в нём множество: его вершины — состояния автомата, а помеченные вершины — принимающие состояния. В префиксном дереве только не хватает переходов по символам, не встречающимся в строках данного множества.

В данном случае, однако, надо распознать не само множество K , а множество всех строк, у которых есть суффикс из K . Для этого надо добавить в префиксное дерево недостающие переходы. Пусть u — вершина, которой соответствует строка $u \in \Sigma^*$, и пусть в префиксном дереве нет перехода из этой вершины по символу $a \in \Sigma$. Тогда у строки ua с помощью функции π можно определить самый длинный её суффикс, являющийся префиксом некоторой строки из K , и определить переход из вершины u по a в состояние, соответствующее этому суффиксу.

Все дуги, входящие в префиксное дерево, становятся переходами конечного автомата.

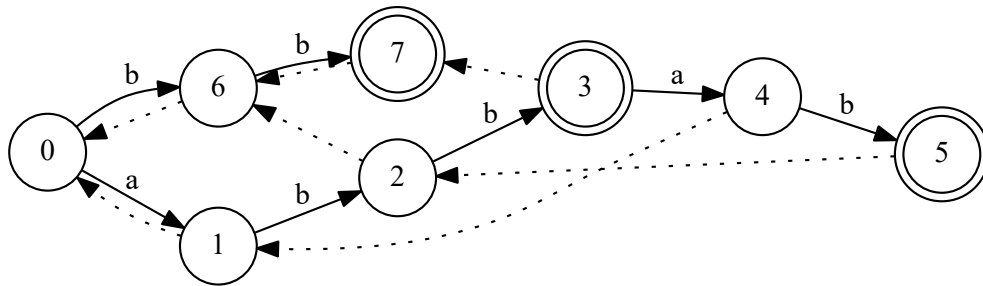
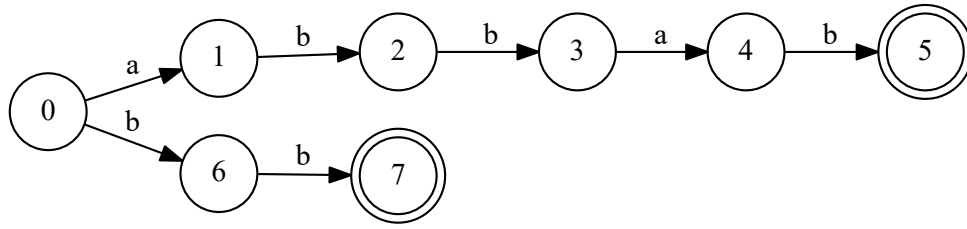


Рис. 7: Построение функции π в алгоритме Ахо–Корасик для множества $K = \{abbab, bb\}$: (сверху) префиксное дерево; (снизу) префиксное дерево с префиксной функцией на нём.

Далее, если в вершине u значение функции переходов $\delta(u, a)$ ещё не определено, то оно определяется процедурой $d(u, a)$, работающей так: если из u есть переход по a , то она возвращает вершину, куда этот переход делается, а если перехода нет, то она запускает $d(\pi(u), a)$ и возвращает то, что та вернёт. Таким образом, опять перебираются суффиксы $\pi(u)$, $\pi(\pi(u))$ и т.д., пока не найдётся переход по a — и $\delta(u, a)$ повторяет этот переход. Как это сделать за линейное время: процедура, вычисляющая $\delta(u, a)$, запоминает результат своей работы в таблице, и когда он потребуется опять, повторные вычисления не производятся.

6 Суффиксные деревья

Суффиксное дерево (Вайнер [1973]) — это особая структура данных, строящаяся по данной строке и обеспечивающая эффективный поиск подстрок в этой строке.

Например, с помощью суффиксного дерева удобно решать задачу поиска в таком варианте: длинная строка («текст») фиксирована, и в ней нужно искать много длинных подстрок — так, чтобы сложность каждого поиска была минимальной. Ранее описанные алгоритмы плохо для этого приспособлены. Однако, получив текст на входе, можно сперва подготовиться к поиску в этом тексте, построив суффиксное дерево для него, так что потом можно станет легко отвечать на запросы с новыми подстроками.

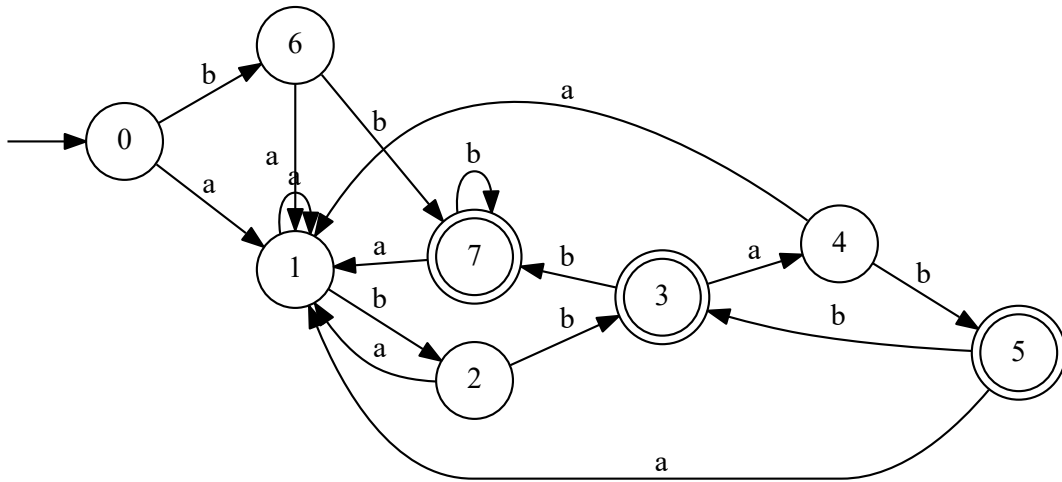


Рис. 8: Конечный автомат, выполняющий алгоритм Ахо–Корасик для множества $K = \{abbab, bb\}$.

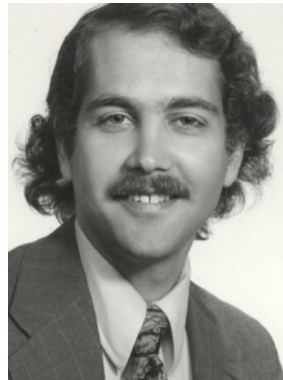


Рис. 9: Питер Вайнер (род. 1942).

6.1 Упрощённое суффиксное дерево

В упрощённом виде, суффиксное дерево — это префиксное дерево для множества суффиксов данной строки. Пусть $w = a_1 \dots a_n$ — очень длинная строка. У неё ровно $n + 1$ суффикс — множество $\text{suffixes}(w) = \{a_1 \dots a_i \mid i \in \{0, \dots, n\}\}$. Если для этого множества построить префиксное дерево, то тогда в строке w будет очень удобно искать подстроки: ведь всякая подстрока w — это начало одного из суффиксов w , и его можно непосредственно найти в префиксном дереве, причём время поиска не будет зависеть от длины текста.

Кроме того, суффиксное дерево содержит *суффиксные ссылки*. Пусть $w = uav$, где $u, v \in \Sigma^*$ и $a \in \Sigma$. Тогда суффиксам av и v соответствуют две вершины в дереве, и в вершине av даётся *суффиксная ссылка* на вершину v , находящуюся на один уровень выше.

Если $w = a_1 \dots a_n$, то, следуя по суффиксным ссылкам из вершины w , можно последовательно обойти все n вершин, соответствующих суффиксам w .

6.2 Построение упрощённого суффиксного дерева

Для данной строки $w = a_1 \dots a_n$ алгоритм последовательно строит суффиксные деревья для строк $w_0 = \varepsilon$, $w_1 = a_1$, $w_2 = a_1 a_2$, и так далее для всех префиксов w . Суффиксное дерево для пустой строки состоит из одной вершины и не содержит суффиксных ссылок. Каждое последующее суффиксное дерево получается из предыдущего его достраиванием.

Пусть дано суффиксное дерево для строки v , а также указатель на вершину, соответствующую всей строке v . Требуется получить суффиксное дерево для строки va , где $a \in \Sigma$. Для этого необходимо продолжить вершины, соответствующие всем суффиксам строки v , символом a . Вершины, соответствующие всем суффиксам строки v , можно просмотреть, следуя по суффиксным ссылкам вплоть до возвращения в корень. Каждую из них следует продолжить символом a , и полученные вершины связать в цепочку новыми суффиксными ссылками.

Реализуется это с помощью рекурсивной процедуры *продолжение*(x, a), получающей в качестве аргумента вершину x , продлевающей её до xa , делающей то же самое со всеми её суффиксами и наконец возвращающей вершину xa . Тогда вызов $v = \text{продолжение}(v, a)$ вычисляет все необходимые продолжения.

Алгоритм 6 Построение упрощённого суффиксного дерева

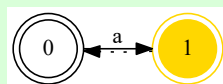
Дано: $w = a_1 \dots a_n$.

- 1: создать вершину ε
- 2: $v = \varepsilon$
- 3: **for** $i = 1, \dots, n$ **do**
- 4: $v = \text{продолжение}(v, a_i)$

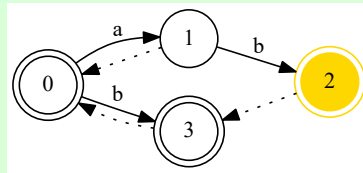
процедура *продолжение*(v, a)

- 1: **if** из v ещё нет перехода по a **then**
 - 2: создать вершину va , соединить v с va
 - 3: **if** $v \neq \varepsilon$ **then**
 - 4: пусть $u = \text{суффиксная_ссылка}[v]$
 - 5: пусть $ua = \text{продолжение}(u, a)$
 - 6: $\text{суффиксная_ссылка}[va] = ua$
 - 7: **else**
 - 8: $\text{суффиксная_ссылка}[va] = \varepsilon$
 - 9: **return** va
-

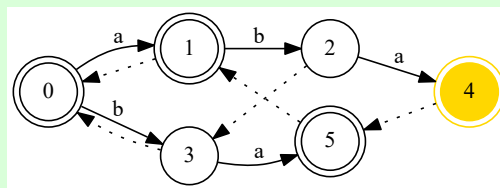
Пример 3 (Построение упрощённого суффиксного дерева для $w = abab$). После чтения первого символа a .



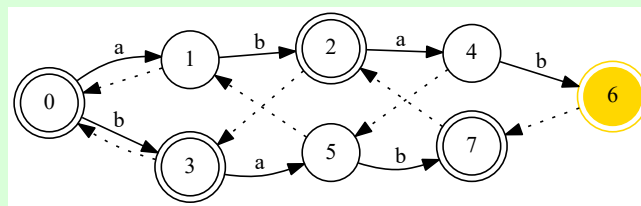
Чтение второго символа b : по суффиксным ссылкам находятся вершины 1 и 0, каждая продлевается по b (в 2 и 3, соответственно), вершины 2, 3, 0 соединяются новой цепочкой суффиксных ссылок.



Чтение третьего символа a : по суффиксным ссылкам —
 2, 3, 0, продлеваются в 4, 5, 1, цепочка суффиксных ссылок.



Чтение четвёртого символа b : по суффиксным ссылкам —
 4, 5, 1, 0, продлеваются в 6, 7, 2, 3, цепочка суффиксных ссылок.



Общий недостаток: размер префиксного дерева для n строк составит $O(n^2)$, и эта верхняя оценка в худшем случае достигается.

Пример 4. Префиксное дерево для множества суффиксов строки $w = a^n b^n$ содержит $(n + 1)^2$ вершин. Дерево для $n = 3$ изображено на рис. 10(сверху).

Это слишком много! И как его ни строй, это будет работать слишком медленно.

Список литературы

- [1975] A. V. Aho, M. J. Corasick, “Efficient string matching: An aid to bibliographic search”, *Communications of the ACM*, 18:6 (1975), 333–340.
- [1959] R. de la Briandais, “File searching using variable length keys”, *Western joint computer conference (IRE-AIEE-ACM’59, San Francisco, 3–5 March 1959)*, 295–298.
- [1987] R. M. Karp, M. O. Rabin, “Efficient randomized pattern-matching algorithms”, *IBM Journal of Research and Development*, 31:2 (1987), 249–260.

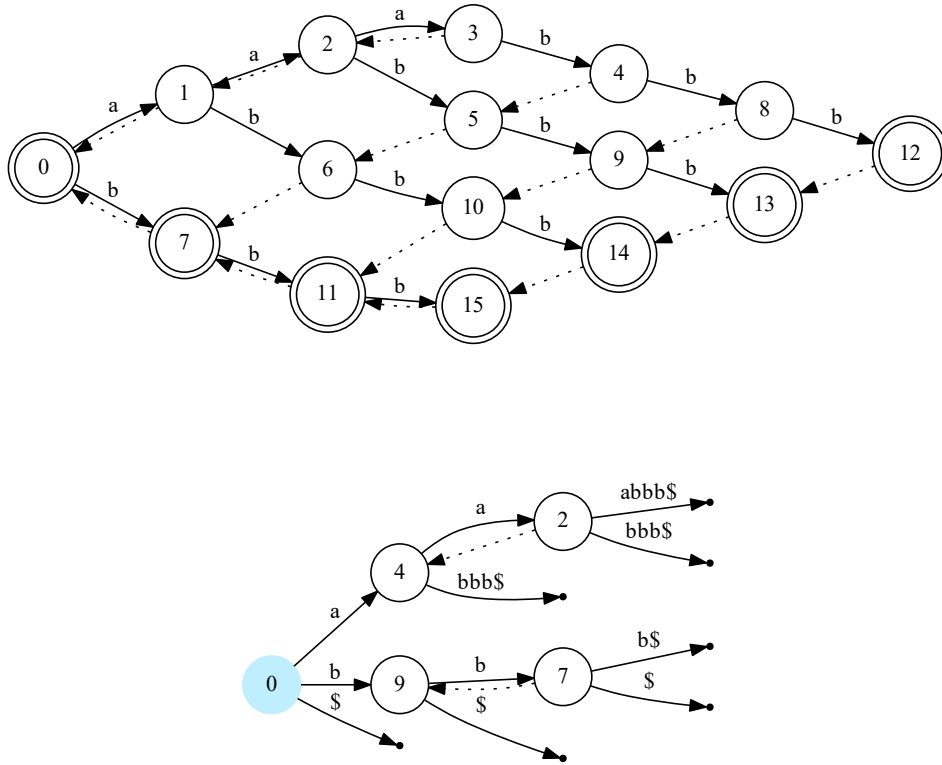


Рис. 10: (сверху) Упрощённое суффиксное дерево для строки $w = aaabbb$; (снизу) суффиксное дерево для $w\$$.

- [1977] D. E. Knuth, J. H. Morris Jr., V. R. Pratt, “Fast pattern matching in strings”, *SIAM Journal on Computing*, 6:2 (1977), 323–350.
- [1973] P. Weiner, “Linear pattern matching algorithms”, *SWAT 1973*, 1–11.